

Models *e Rules*

Duas escolhas definem como o agente trabalha para você: **qual modelo pensa** em cada pedido e **o que ele sabe** sobre você e o seu projeto antes de começar. Nesta aula: os modelos disponíveis no Cursor, como cada um é cobrado, o Auto, o esforço, e as **rules** (de usuário, de projeto, de equipe e o `AGENTS.md`).

Cursor Models · Other Models

Composer · Grok · Claude · GPT · Gemini

Auto e Cursor Router

Esforço e Fast

Rules .mdc

AGENTS.md

Formação Claude Code 2026 · Baseado em [cursor.com](#) (Docs: Models & Pricing, Cursor Router, páginas de cada modelo, Rules; Help: Available models, Rules, Max Mode), consultados em outubro de 2026 · Telas: reprodução ilustrativa

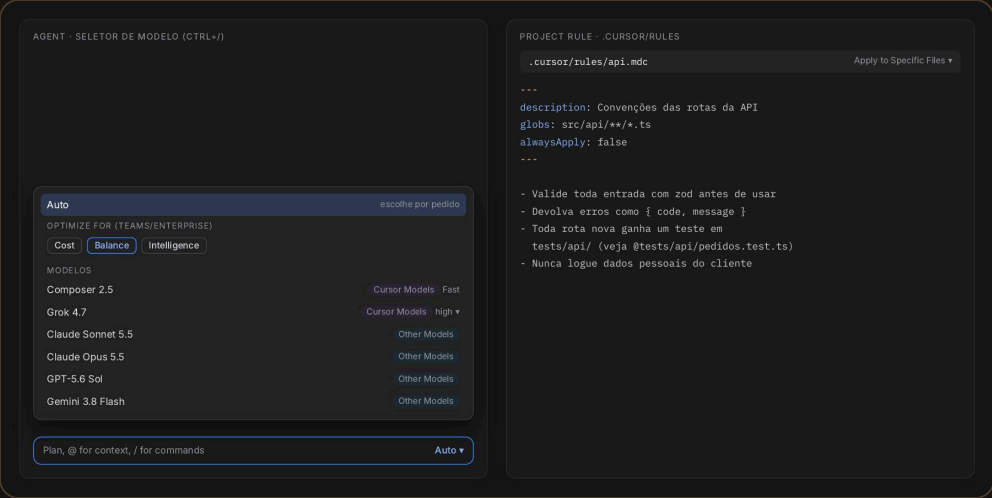
Quem pensa *e o que ele sabe*

MODEL · QUEM PENSA

O modelo de linguagem que lê o pedido, decide os passos e escreve o código. Muda a **qualidade**, a **velocidade** e o **custo** de cada pedido. Dá para trocar a qualquer momento.

RULES · O QUE ELE SABE

Modelos não guardam memória entre conversas. As rules são **instruções permanentes**, incluídas no início do contexto, para o agente seguir seu estilo, seus padrões e seus fluxos sem você repetir.



Seletor de modelos e um arquivo de rule · reprodução ilustrativa

Dois bolsos *de uso*

Nos planos Pro, Pro+ e Ultra, o uso incluído é dividido em **dois bolsos**, cada um renovado no ciclo mensal. O modelo escolhido decide de qual bolso sai e com que rapidez ele acaba.

CURSOR MODELS

Composer 2.5 e a família **Grok 4.5, 4.6 e 4.7** (treinada pelo Cursor com a SpaceXAI). Têm **bem mais uso incluído**. Também é o único bolso do plano Start (Índia).

OTHER MODELS

Modelos de terceiros (Anthropic, OpenAI, Google, Meta e outros), cobrados ao **preço da API** de cada um. Ao esgotar, dá para seguir com uso sob demanda, aos mesmos preços.

Os dois bolsos aparecem nas configurações do editor e no painel cursor.com/dashboard/usage.

Nos planos **Teams e Enterprise**, pedidos a modelos de terceiros têm ainda a taxa do Cursor: US\$ 0,25 por milhão de tokens. Os modelos próprios do Cursor ficam isentos.

Os modelos *do Cursor*

COMPOSER 2.5

- Modelo próprio do Cursor, ajustado para usar ferramentas, editar arquivos e operar o terminal dentro do Cursor.
- Forte em tarefas longas de agente; melhor calibragem de esforço e entendimento da intenção que o Composer 2.
- A variante **Fast** é a padrão no produto (mesma inteligência, mais rápida).
- Preço: US\$ 0,50 entrada / US\$ 2,50 saída por milhão de tokens; Fast: US\$ 3 / US\$ 15.

GROK 4.7

- Modelo de ponta do Cursor com a SpaceXAI, para o trabalho mais difícil e demorado. Confere melhor os próprios resultados que o 4.6.
- Contexto de 256 mil tokens, ou 500 mil no modo de contexto longo (cobrado em dobro).
- **Quatro níveis de esforço:** xhigh, high (padrão), medium e low.
- Preço: US\$ 2 / US\$ 6; Fast (padrão no Pro e acima): US\$ 4 / US\$ 12.

A própria ajuda do Cursor resume: **Grok 4.7** é o carro-chefe para as tarefas mais difíceis e longas; **Composer** é o modelo rápido e econômico, capaz na maioria das tarefas e feito para programar de forma interativa.

Os modelos *de terceiros*

MODELO	PONTO FORTE (SEGUNDO O CURSOR)	ATENÇÃO
Claude Opus 5.5	Maior nota no CursorBench no lançamento; comunicação clara; forte em interface e como coordenador de subagentes	Preço de Opus: gasta o bolso mais rápido que Sonnet ou Composer
Claude Sonnet 5.5	Resultados de ponta a preço de Sonnet; melhora com mais esforço; contexto de até 1 milhão	Esforço máximo custa cerca de 6 vezes mais por tarefa
Claude Fable 5.1	Trabalho longo e autônomo, raciocínio profundo	A Anthropic retém dados: com Privacy Mode, exige aprovação
GPT-5.6 Sol	O mais forte da família: persistente em sessões longas, direto e conciso	Preço mais alto; pode usar subagentes demais
GPT-5.6 Terra · Luna	Terra: o dia a dia, mais barato que o Sol. Luna: o mais barato, bom para protótipos e subagentes	Não são o topo para as tarefas mais difíceis
Gemini 3.1 Pro	Lê imagens junto com o código: bom para interface a partir de mockups; contexto de 1 milhão	Preço sobe acima de 200 mil tokens de entrada
Gemini 3.8 Flash	Rápido e barato, com contexto de 1 milhão e três níveis de esforço	Escolha velocidade, não o máximo de raciocínio
Muse Spark 1.3 (Meta)	Modelo de ponta da Meta, com seis níveis de esforço e preço baixo	Provedor novo no Cursor

Muitos outros modelos (versões anteriores de Claude, GPT, Gemini, além de GLM e Kimi) existem, mas ficam ocultos por padrão.

Quanto custa *cada um*

MODELO	BOLSO	ENTRADA	SAÍDA	LEITURA DE CACHE
Composer 2.5	Cursor Models	US\$ 0,50	US\$ 2,50	US\$ 0,20
Grok 4.7	Cursor Models	US\$ 2	US\$ 6	US\$ 0,50
GPT-5.6 Luna	Other Models	US\$ 0,20	US\$ 1,20	US\$ 0,02
Gemini 3.8 Flash	Other Models	US\$ 0,75	US\$ 3,50	US\$ 0,075
Muse Spark 1.3	Other Models	US\$ 1,25	US\$ 4,25	US\$ 0,15
Claude Sonnet 5.5	Other Models	US\$ 2	US\$ 10	US\$ 0,20
Gemini 3.1 Pro	Other Models	US\$ 2	US\$ 12	US\$ 0,20
GPT-5.6 Terra	Other Models	US\$ 2	US\$ 12	US\$ 0,20
Claude Opus 5.5	Other Models	US\$ 4	US\$ 20	US\$ 0,20
GPT-5.6 Sol	Other Models	US\$ 4	US\$ 20	US\$ 0,40
Claude Fable 5.1	Other Models	US\$ 10	US\$ 50	US\$ 0,25

Preços por milhão de tokens, conforme cursor.com/docs/models-and-pricing em outubro de 2026. A saída (o que o modelo escreve) costuma pesar mais que a entrada. Planos e limites de uso: veja a aula 01 do módulo 34 e cursor.com/pricing.

Auto e o Cursor Router

Com **Auto**, você não escolhe o modelo: o Cursor escolhe a cada pedido, equilibrando inteligência, custo e confiabilidade. É o recomendado para o dia a dia.

CURSOR ROUTER (TEAMS E ENTERPRISE)

Nesses planos, o Auto usa um roteador: um classificador manda os pedidos simples para modelos rápidos e os complexos para os mais capazes. Você escolhe a otimização no seletor:

- **Cost:** economiza tokens.
- **Balance:** equilibra inteligência, velocidade e custo.
- **Intelligence:** os mais capazes para tarefas difíceis.

- ✓ Todo modo do Auto é cobrado ao **preço do modelo escolhido** pelo roteador
- ✓ Balance e Intelligence gastam o limite mais rápido que Cost
- ✓ Por padrão, o nome do modelo usado fica oculto, para você julgar o resultado e não a marca (o admin pode mostrar)
- ✓ O admin pode impor o Auto: **Soft** (padrão em conversa nova) ou **Hard** (trava o seletor)

O plano Start (Índia) não inclui o Auto.

Esforço, Fast *e contexto*

ESFORÇO

Vários modelos aceitam níveis de esforço: mais esforço = mais tempo pensando, mais tokens, melhor resultado em problemas difíceis. Ex.: Grok 4.7 (low a xhigh), Gemini 3.8 Flash (low, medium, high), Muse Spark (minimal a max).

FAST

Variante mais rápida do mesmo modelo, geralmente com preço maior (no Grok 4.7, o dobro). No Composer 2.5 e no Grok 4.7 (Pro e acima), o Fast já é o padrão.

CONTEXTO

Quanto o modelo consegue "ver" de uma vez. Vários chegam a 1 milhão de tokens; alguns cobram mais acima de um limite (ex.: Gemini 3.1 Pro acima de 200 mil; Grok 4.7 acima de 256 mil).

Regra prática: comece no esforço padrão. Suba só quando o resultado não vier bom; desça para tarefas simples e repetitivas.

Max Mode existe só nos planos antigos, cobrados por pedido: amplia o contexto, a preço de API + 20%. Nos planos atuais, você não precisa dele.

Qual modelo *para cada tarefa*

SITUAÇÃO	SUGESTÃO	POR QUÊ
Dia a dia, sem pensar no assunto	Auto	O Cursor escolhe por pedido
Edições rápidas, tarefas rotineiras	Composer 2.5	Rápido, barato e com mais uso incluído
Tarefa longa e difícil, dentro do bolso do Cursor	Grok 4.7 (high ou xhigh)	Carro-chefe do Cursor, com uso incluído maior
Refatoração complexa, interface, coordenar subagentes	Claude Opus 5.5	Melhor nota no CursorBench no lançamento
Qualidade alta com custo menor	Claude Sonnet 5.5 ou GPT-5.6 Terra	Bom equilíbrio entre preço e resultado
Planejar, depurar conversando, sessões de horas	GPT-5.6 Sol	Persistente e direto
Interface a partir de um print ou mockup	Gemini 3.1 Pro	Lê imagens junto com o código
Muito volume, subagentes, protótipos	GPT-5.6 Luna ou Gemini 3.8 Flash	Os mais baratos

Dá para trocar no meio da conversa: explore com um modelo rápido e implemente com um mais capaz.

Onde escolher *e ajustar*

QUERO	ONDE
Trocar o modelo da conversa	Seletor no topo da caixa de mensagem, ou <code>Ctrl</code> + <code>/</code> para alternar (Mac: <code>Cmd</code> + <code>/</code>). A escolha vale para as próximas conversas até você mudar
Definir o padrão	Cursor Settings > Models
Ver modelos ocultos	Cursor Settings > Models, ativando os que você quer ver no seletor
Ajustar esforço ou Fast	No próprio seletor, nos modelos que oferecem
Usar a minha chave de API	Cursor Settings > Models > provedor > colar a chave > Save
Liberar o Fable com Privacy Mode	Aprovar o modelo no painel da equipe ou da conta (Team Settings)

Um modelo pode não aparecer na sua região por restrição do provedor. Nesse caso, use Auto ou outro modelo.

Quatro tipos *de rules*

TIPO	ONDE FICA	VALE PARA
Project Rules	<code>.cursor/rules/*.mdc</code> , versionadas no Git	Aquele projeto, para toda a equipe
User Rules	Customize > Rules; sincronizam com a sua conta	Todos os seus projetos, em qualquer computador
Team Rules	Painel da equipe (Teams e Enterprise)	Todos os projetos da organização; podem ser obrigatórias
AGENTS.md	Arquivo Markdown na raiz (e em subpastas)	Aquele projeto, do jeito mais simples

Precedência: Team Rules → Project Rules → User Rules. Todas são somadas; se houver conflito, vale a que vem antes.

Alcance: as rules valem para o Agent em todos os modos. Não afetam o **Tab**, e as User Rules não valem na edição inline (`Ctrl` + `K`).

Anatomia de *uma project rule*

Cada rule é um arquivo `.mdc` em `.cursor/rules/`: um cabeçalho (frontmatter) e as instruções em Markdown. Um `.md` comum nessa pasta é **ignorado**.

TIPO (NO MENU)	CABEÇALHO	QUANDO ENTRA
Always Apply	<code>alwaysApply: true</code>	Em toda conversa
Apply to Specific Files	<code>globs: ...</code>	Quando um arquivo que casa está no contexto
Apply Intelligently	<code>description: ...</code>	O agente lê a descrição e decide
Apply Manually	nenhum dos três	Só com <code>@nome-da-rule</code>

```
# .cursor/rules/componentes.mdc
---
globs: src/components/**/*.*tsx
alwaysApply: false
---
- Use exports nomeados, não default
- Estilos num arquivo .module.css ao lado
- Componentes com menos de 200 linhas
- Prefira composição a repassar props
  por várias camadas
```

Rule "Apply Intelligently" sem `description` ou "Apply to Specific Files" com padrão que não casa: são as causas mais comuns de "minha rule não funciona".

Padrões de arquivo *(globs)*

PADRÃO	CASA COM
<code>*.ts</code>	Arquivos <code>.ts</code> na raiz
<code>**/*.ts</code>	Arquivos <code>.ts</code> em qualquer pasta
<code>src/**</code>	Tudo dentro de <code>src/</code>
<code>src/**/*.tsx</code>	Todo <code>.tsx</code> dentro de <code>src/</code>
<code>docs/**/*.md,</code> <code>docs/**/*.mdx</code>	Markdown em <code>docs/</code> (vírgula separa padrões)
<code>tailwind.config.*</code>	<code>tailwind.config</code> com qualquer extensão

- `*` casa com um trecho do nome; `**` atravessa pastas.
- Separe vários padrões com vírgula.
- Dentro da rule, use `@arquivo.ts` para apontar um exemplo em vez de copiar o código: a rule fica curta e não desatualiza.

Rules por arquivo economizam contexto: só entram quando aquele tipo de arquivo está em jogo, ao contrário das "Always Apply".

Três jeitos *de criar uma rule*

PEDINDO AO AGENTE

Digite `/create-rule` e descreva o que quer. O agente cria o arquivo com o cabeçalho certo em `.cursor/rules`. Também dá para pedir em texto livre.

PELO CUSTOMIZE

Barra lateral > **Customize** > **Rules** > **Add Rule**. Ali você também vê todas as rules e se estão ativas.

PELA PALETA

`Ctrl` + `Shift` + `P` > **New Cursor Rule**, dê um nome (ex.: `react-patterns`), escreva e escolha o tipo no menu.

USER RULE DE EXEMPLO

Responda sempre em português do Brasil, de forma concisa. Mantenha nomes de código e comandos no original. Antes de mudar algo grande, mostre o plano.

Viu o agente errar a mesma coisa duas vezes? É hora de uma rule. No GitHub, dá até para marcar `@cursor` numa issue ou PR para o agente atualizar a rule.

AGENTS.md: *o jeito simples*

Um arquivo Markdown comum, sem cabeçalho, na raiz do projeto. O Cursor lê automaticamente. É um formato reconhecido por vários agentes, então serve também fora do Cursor.

```
# AGENTS.md
# Instruções do projeto
- Use TypeScript em todo arquivo novo
- Acesso a dados sempre pelo padrão repository
- Componentes com menos de 200 linhas
- Rode `npm test` antes de dar a tarefa por concluída
```

AGENTS.MD ANINHADO

Coloque outros `AGENTS.md` em subpastas. Eles valem quando o agente trabalha naqueles arquivos, somados aos das pastas acima; o mais específico tem prioridade.

```
projeto/
  AGENTS.md           # geral
  frontend/AGENTS.md  # só frontend
  backend/AGENTS.md   # só backend
```

Precisa escolher **quando** cada instrução entra (por arquivo, por descrição, manual)? Use as rules `.mdc`.

Rules que *funcionam*

FAÇA

- ✓ Menos de 500 linhas por rule; divida as grandes em várias pequenas
- ✓ Uma pasta `.cursor/rules` simples, um assunto por arquivo, nomes claros
- ✓ Exemplos concretos ou referência a arquivos (`@arquivo`)
- ✓ Escreva como documentação interna: direto e acionável
- ✓ Comece pequeno e acrescente quando o agente repetir um erro
- ✓ Versione no Git para a equipe aproveitar

EVITE

- Copiar guias de estilo inteiros: use um linter
- Explicar comandos que o agente já conhece (npm, git, pytest)
- Regras para casos raros
- Duplicar código que já existe no projeto: aponte o exemplo
- Usar rules como único controle de segurança

Para levar rules de um repositório para outro, o Cursor não importa rules soltas: empacote-as num **plugin** e publique num marketplace.

Um kit inicial *para o seu projeto*

ARQUIVO	TIPO	CONTEÚDO
User Rule	Usuário	Idioma, tom das respostas, "mostre o plano antes de mudanças grandes"
AGENTS.md	Projeto	Visão geral: stack, como rodar e testar, onde fica cada coisa
.cursor/rules/testes.mdc	Always Apply	"Toda mudança vem com teste; rode a suíte antes de concluir"
.cursor/rules/frontend.mdc	globs: src/components/**	Padrões de componentes, estilos, acessibilidade
.cursor/rules/api.mdc	globs: src/api/**	Validação de entrada, formato de erro, testes da rota
.cursor/rules/migracoes.mdc	Manual	Como escrever migrações (chamada com @migracoes)

Teste rápido: peça ao agente "quais rules estão valendo nesta conversa?" e depois abra o anel de contexto: a categoria *Rules* mostra quanto elas ocupam.

Teste o que *ficou*

1. De qual bolso sai o uso do Composer 2.5?

A) Other Models

B) Cursor Models, que tem bem mais uso incluído

C) É sempre grátis

D) Só sob demanda

B. Junto com a família Grok.

2. Qual a ordem de precedência das rules?

A) User → Project → Team

B) Team → Project → User

C) Project → User → Team

D) Não há ordem

B. Somam-se; no conflito vale a que vem antes.

3. Uma rule só com `globs` e `alwaysApply: false` entra quando?

A) Sempre

B) Quando um arquivo que casa com o padrão está no contexto

C) Nunca

D) Só com @

B. É a "Apply to Specific Files".

4. Onde se definem as User Rules?

A) Em `.cursor/rules`

B) Em Customize > Rules

C) No `package.json`

D) No terminal

B. E sincronizam com a conta.

○ RECAPITULANDO

O modelo certo, *com as instruções certas*

O uso incluído vem em dois bolsos: **Cursor Models** (Composer 2.5 e Grok, com mais uso) e **Other Models** (Claude, GPT-5.6, Gemini, Muse, ao preço da API). **Auto** escolhe por você; esforço, Fast e contexto mudam custo e resultado. As **rules** dão memória ao agente: User Rules em Customize, Project Rules em `.cursor/rules/*.mdc` com quatro modos de aplicação, Team Rules para a organização e o `AGENTS.md` como caminho simples.

Ctrl+/
↵

Auto · Cost · Balance · Intelligence

Composer · Grok · Opus · Sol

alwaysApply · globs · description

/create-rule
↵

AGENTS.md
↵

Próxima aula → Abrindo Projetos

Fontes: cursor.com/docs (Models & Pricing, Cursor Router, Composer 2.5, Grok 4.7, Claude Opus 5.5, Sonnet 5.5, Fable 5.1, GPT-5.6 Sol, Terra e Luna, Gemini 3.1 Pro e 3.8 Flash, Muse Spark 1.3, Rules) e cursor.com/help (Available models, Rules, Max Mode on legacy plans), consultados em outubro de 2026. Modelos e preços mudam com frequência.